Security Hardening

What the Pi Zero 2 W build and the common tools do to keep a node safe: how people log in, how a request is checked before it runs, which headers each page sends, which requests are refused, and where the protection stops.

LOGIN | REQUEST CHECKS | HEADERS | GAPS

SUMMARY · READ-ONLY REVIEW

Part 1 of 3. Sources: the pi02w tools (nodecontrol, sysmon, wifimon, 44helper), the common tools (instmon, pibrary, Node Talker, Node Recorder) on branch ccr-9d4fe19f-ssud83. Nothing was run on a node.

How a person gets past the login

1.1 The root password

The Dashboard (nodecontrol) checks the root password first through the system's PAM stack, using the `login` service. If PAM is not usable, it falls back to reading the root hash from `/etc/shadow` and checking it with `crypt(3)` from libcrypt. The logging of the password itself was not traced in this review.

1.2 Lockouts

A failed login counts against the address it came from. Two rules are in use:

TOOLS	RULE
Dashboard, SysMon, 44helper	Five failures within five minutes lock the address out for 15 minutes (`_LOGIN_LOCKOUT_SEC = 15 * 60`)
instmon, wifimon, Node Talker	Each failure lengthens the wait. The first wait is 5 seconds, doubling, and capped at 300 seconds. A locked request is answered with 429 and a `Retry-After` header

The lockout is keyed on the client address the server sees (`client_address`). A proxy in front of the node would put every user behind one address, so the proxy rules in Part 5 matter here too.

1.3 Session cookies

After a good login the server sets a session cookie. Each tool sets its own name (for example `helper44_session` in 44helper, `asl_dvs_session` elsewhere). The cookie is sent with `HttpOnly`, `SameSite=Strict`, `Path=/` and a `Max-Age`. A script on the page cannot read it, and a browser does not send it on a request that starts on another site.

Refusing requests that look forged

A web page that a person has open can be made to send a request on their behalf. The tools block that in two ways: they demand a header that a plain web form cannot send, or they demand a content type that a plain form cannot send.

CHECK	WHERE	REFUSED WITH
`Content-Type: application/json` **required**	Dashboard, SysMon	415
`X-Requested-With: XMLHttpRequest` **required**	44helper, instmon, wifimon, pibrary, Node Talker, Node Recorder, amp Node Talker	Refused. The exact status code varies by tool and was not tabulated

2.1 Why these work

A browser form can send a plain POST with a few content types, and it cannot add a custom header to a request from another site without a permission check that fails. Requiring either one closes the common route.

The check is only as good as its coverage. The Dashboard and SysMon use the content-type check, and the other tools use the header check. A new endpoint added without the check has no protection, so each new route needs the same guard.

The local key and the loopback-only API

3.1 Loopback peers only

The Dashboard's local API accepts a request only from a loopback address (`127.0.0.1`, `::1`, or the IPv4-mapped form). Any other peer is refused, even if it has the key.

3.2 The key

The local API also needs the header `X-ASL-DVS-Local-Key`. The key is kept in `/etc/asl_dvs/local_api.key`, mode 0600, and must be at least 32 hexadecimal characters. A key file that does not meet that rule is not used.

The name of the header and the path keep the `asl_dvs` system name, as the naming rules require.

What each page tells the browser

HEADER	PURPOSE	SET BY (FOUND IN SOURCE)
`X-Frame-Options: DENY`	Stops the page being shown inside another site's frame (clickjacking)	44helper, wifimon
`X-Content-Type-Options: nosniff`	Stops the browser guessing a file's type	Dashboard, 44helper, instmon, wifimon
`Referrer-Policy: no-referrer`	The page does not send the address it came from	44helper, instmon, wifimon
`Content-Security-Policy`	Limits which scripts, styles and connections the page may use	Dashboard (per page), 44helper, wifimon

A search of the SysMon pi02w file found none of these four headers. SysMon's pages therefore rely on its other checks (the content-type check in Part 2 and the login) and not on these headers. This is a gap to confirm on a running node before the book states it.

Requests that came through a proxy or tunnel

A request that passes through a reverse proxy or a tunnel such as Cloudflare carries a header that names the proxy. Several tools refuse those requests on purpose, so a remote visitor cannot reach a control through a tunnel that puts them on the local network.

TOOL	HEADERS CHECKED	EFFECT
wifimon	X-Forwarded-For, X-Forwarded-Host, X-Forwarded-Proto, X-Real-Ip, Forwarded, Via, Cf-Connecting-Ip, Cf-Ray, Cf-Visitor, Cf-Ipcountry	Control actions refused with 403
Node Recorder	Forwarded, X-Forwarded-For, X-Real-IP, CF-Connecting-IP, Tailscale-Funnel-Request	Refused as proxied
44helper terminal	Cf-Connecting-Ip, Cf-Ray, Cf-Ipcountry, Cf-Visitor, X-Forwarded-For and others	Terminal refused with a message that it works only from the local network
Node Talker	Its own proxy list, checked with the LAN test	Proxied requests are treated as coming from the internet

The lists are not the same in every tool. A header missing from one list is not checked there. The lists are best read as a tripwire, and the book should not present them as a complete defence against a tunnel.

Plain HTTP, root services and the terminal

6.1 Plain HTTP by default

- **wifimon** is set to plain HTTP (`USE_TLS = False`) and binds to `0.0.0.0`, so it listens on every interface.
- **Node Talker** and **Node Recorder** default to HTTPS (`https: True`).
- **The Dashboard** binds to `0.0.0.0`.

Plain HTTP means a login password and a session cookie cross the local network unencrypted. On a home LAN that is a known risk, and the book should state it rather than imply the logins are encrypted.

6.2 Services run as root

The 44helper unit file runs the service as `User=root`. The other unit files in the pi02w and common tools set no sandboxing directives (no `NoNewPrivileges`, `ProtectSystem` or `PrivateTmp`). A flaw in any of these web services therefore has root on the node.

6.3 The terminal is a root shell

The 44helper terminal opens a shell in the browser. Before it opens, the server checks that the request is not proxied, that it comes from loopback, link-local or a private address, that the service runs as root, and that the terminal-enable switch is set. The terminal is the most sensitive control in the pi02w build, and the book should describe it as root access gated by the network and the switch.

The 44helper job worker runs commands through a shell (`shell=True`). The tab actions are limited to an exact command list (`_dispatch_tab_action`), and the book should note that the safety of that step rests on the list.

What the full build has that the pi02w build does not

The full build of SysMon (`sysmon_v6_13_95_20261028.py`) includes a security audit with 11 checks, among them the Asterisk and AMI default credentials, the Dashboard web authentication, SSH hardening, Fail2ban jails, service user, unit file permissions and Asterisk file permissions. The pi02w build of SysMon (`sysmon_pi02w_v6_13_83_20261028.py`) has no audit table.

This summary changes no code. The gap is recorded here so the book does not describe the audit as a feature of both builds. Porting the audit to the pi02w build is a separate change, and under the repo rules it needs a request that names the pi02w build.

What the hardening does and does not cover

8.1 Layers, not one switch

Login, lockout, the request checks, the local key, the header set and the proxy refusals each stop a different route. None of them is a complete defence alone, and each tool sets a different subset.

8.2 The LAN is the outer wall

Most controls assume the node is on a trusted home or office LAN. Any device on that LAN can reach the plain HTTP ports. The LAN-only policy (the second summary in this series) describes that wall.

8.3 Not checked

- The PAM configuration of the `login` service on a running node.
- Every endpoint in every tool for the request checks in Part 2. Only the check functions were read.
- The full Node Talker proxy list (only its name was read).
- Whether a browser on a real node sends the headers listed in Part 4.